Carga de librerias

- link https://colab.research.google.com/github/fandinoia1980-web/ALGORITMOS_DE_OPTIMIZACION/blob/main/Algoritmos_AG3.ipynb
- Github: https://github.com/fandinoia1980-web/ALGORITMOS_DE_OPTIMIZACION.git

In [2]:
```
!pip install requests    # Hacer llamadas http a paginas de la red
!pip install tsplib95    # Modulo para las instancias del problema del TSP
```

```
ERROR: Invalid requirement: '#': Expected package name at the start of dependency
specifier
    #
    ^
ERROR: Invalid requirement: '#': Expected package name at the start of dependency
specifier
    #
    ^
```

Carga de los datos del problema

In [1]:
```
import urllib.request #Hacer llamadas http a paginas de la red
import tsplib95       #Modulo para las instancias del problema del TSP
import math           #Modulo de funciones matematicas. Se usa para exp
import random         #Para generar valores aleatorios


#http://elib.zib.de/pub/mp-testdata/tsp/tsplib/
#Documentacion :
  # http://comopt.ifi.uni-heidelberg.de/software/TSPLIB95/tsp95.pdf
  # https://tsplib95.readthedocs.io/en/stable/pages/usage.html
  # https://tsplib95.readthedocs.io/en/v0.6.1/modules.html
  # https://pypi.org/project/tsplib95/

#Descargamos el fichero de datos(Matriz de distancias)
file = "swiss42.tsp" ;
urllib.request.urlretrieve("http://comopt.ifi.uni-heidelberg.de/software/TSPLIB9
!gzip -d swiss42.tsp.gz     #Descomprimir el fichero de datos

#Coordendas 51-city problem (Christofides/Eilon)
#file = "eil51.tsp" ; urllib.request.urlretrieve("http://comopt.ifi.uni-heidelbe

#Coordenadas - 48 capitals of the US (Padberg/Rinaldi)
#file = "att48.tsp" ; urllib.request.urlretrieve("http://comopt.ifi.uni-heidelbe
```

```
---------------------------------------------------------------------------
TimeoutError                              Traceback (most recent call last)
File ~\anaconda3\envs\PYTHON_IA\Lib\urllib\request.py:1319, in AbstractHTTPHandle
r.do_open(self, http_class, req, **http_conn_args)
   1318 try:
-> 1319     h.request(req.get_method(), req.selector, req.data, headers,
   1320               encode_chunked=req.has_header(                   ))
   1321 except OSError as err: # timeout error

File ~\anaconda3\envs\PYTHON_IA\Lib\http\client.py:1338, in HTTPConnection.reques
t(self, method, url, body, headers, encode_chunked)
   1337 """Send a complete request to the server."""
-> 1338 self._send_request(method, url, body, headers, encode_chunked)

File ~\anaconda3\envs\PYTHON_IA\Lib\http\client.py:1384, in HTTPConnection._send_
request(self, method, url, body, headers, encode_chunked)
   1383     body = _encode(body, 'body')
-> 1384 self.endheaders(body, encode_chunked=encode_chunked)

File ~\anaconda3\envs\PYTHON_IA\Lib\http\client.py:1333, in HTTPConnection.endhea
ders(self, message_body, encode_chunked)
   1332     raise CannotSendHeader()
-> 1333 self._send_output(message_body, encode_chunked=encode_chunked)

File ~\anaconda3\envs\PYTHON_IA\Lib\http\client.py:1093, in HTTPConnection._send_
output(self, message_body, encode_chunked)
   1092 del self._buffer[:]
-> 1093 self.send(msg)
   1095 if message_body is not None:
   1096
   1097     # create a consistent interface to message_body

File ~\anaconda3\envs\PYTHON_IA\Lib\http\client.py:1037, in HTTPConnection.send(s
elf, data)
   1036 if self.auto_open:
-> 1037     self.connect()
   1038 else:

File ~\anaconda3\envs\PYTHON_IA\Lib\http\client.py:1003, in HTTPConnection.connec
t(self)
   1002 sys.audit("http.client.connect", self, self.host, self.port)
-> 1003 self.sock = self._create_connection(
   1004     (self.host,self.port), self.timeout, self.source_address)
   1005 # Might fail in OSs that don't implement TCP_NODELAY

File ~\anaconda3\envs\PYTHON_IA\Lib\socket.py:864, in create_connection(address,
timeout, source_address, all_errors)
    863 if not all_errors:
--> 864     raise exceptions[0]
    865 raise ExceptionGroup("create_connection failed", exceptions)

File ~\anaconda3\envs\PYTHON_IA\Lib\socket.py:849, in create_connection(address,
timeout, source_address, all_errors)
    848     sock.bind(source_address)
--> 849 sock.connect(sa)
    850 # Break explicitly a reference cycle

TimeoutError: [WinError 10060] Se produjo un error durante el intento de conexión
ya que la parte conectada no respondió adecuadamente tras un periodo de tiempo, o
bien se produjo un error en la conexión establecida ya que el host conectado no h
```

```
a podido responder

During handling of the above exception, another exception occurred:

URLError                                  Traceback (most recent call last)
Cell In[1], line 16
      7 #http://elib.zib.de/pub/mp-testdata/tsp/tsplib/
      8 #Documentacion :
      9   # http://comopt.ifi.uni-heidelberg.de/software/TSPLIB95/tsp95.pdf
   (...)     13
     14 #Descargamos el fichero de datos(Matriz de distancias)
     15 file = "swiss42.tsp" ;
---> 16 urllib.request.urlretrieve(
                                , file +      )
     17 get_ipython().system('gzip -d swiss42.tsp.gz     #Descomprimir el fichero
de datos')
     19 #Coordendas 51-city problem (Christofides/Eilon)
     20 #file = "eil51.tsp" ; urllib.request.urlretrieve("http://comopt.ifi.uni-h
eidelberg.de/software/TSPLIB95/tsp/eil51.tsp.gz", file)
     21
     22 #Coordenadas - 48 capitals of the US (Padberg/Rinaldi)
     23 #file = "att48.tsp" ; urllib.request.urlretrieve("http://comopt.ifi.uni-h
eidelberg.de/software/TSPLIB95/tsp/att48.tsp.gz", file)

File ~\anaconda3\envs\PYTHON_IA\Lib\urllib\request.py:214, in urlretrieve(url, fi
lename, reporthook, data)
    197 """
    198 Retrieve a URL into a temporary location on disk.
    199
   (...)    210 data file as well as the resulting HTTPMessage object.
    211 """
    212 url_type, path = _splittype(url)
--> 214 with contextlib.closing(urlopen(url, data)) as fp:
    215     headers = fp.info()
    217     # Just return the local path and the "headers" for file://
    218     # URLs. No sense in performing a copy unless requested.

File ~\anaconda3\envs\PYTHON_IA\Lib\urllib\request.py:189, in urlopen(url, data,
timeout, context)
    187 else:
    188     opener = _opener
--> 189 return opener.open(url, data, timeout)

File ~\anaconda3\envs\PYTHON_IA\Lib\urllib\request.py:489, in OpenerDirector.open
(self, fullurl, data, timeout)
    486     req = meth(req)
    488 sys.audit('urllib.Request', req.full_url, req.data, req.headers, req.get_
method())
--> 489 response = self._open(req, data)
    491 # post-process response
    492 meth_name = protocol+"_response"

File ~\anaconda3\envs\PYTHON_IA\Lib\urllib\request.py:506, in OpenerDirector._ope
n(self, req, data)
    503     return result
    505 protocol = req.type
--> 506 result = self._call_chain(self.handle_open, protocol, protocol +
    507                                 , req)
    508 if result:
    509     return result
```

```
File ~\anaconda3\envs\PYTHON_IA\Lib\urllib\request.py:466, in OpenerDirector._cal
l_chain(self, chain, kind, meth_name, *args)
    464 for handler in handlers:
    465     func = getattr(handler, meth_name)
--> 466     result = func(*args)
    467     if result is not None:
    468         return result

File ~\anaconda3\envs\PYTHON_IA\Lib\urllib\request.py:1348, in HTTPHandler.http_o
pen(self, req)
   1347 def http_open(self, req):
-> 1348     return self.do_open(http.client.HTTPConnection, req)

File ~\anaconda3\envs\PYTHON_IA\Lib\urllib\request.py:1322, in AbstractHTTPHandle
r.do_open(self, http_class, req, **http_conn_args)
   1319         h.request(req.get_method(), req.selector, req.data, headers,
   1320                   encode_chunked=req.has_header('Transfer-encoding'))
   1321     except OSError as err: # timeout error
-> 1322         raise URLError(err)
   1323     r = h.getresponse()
   1324 except:

URLError: <urlopen error [WinError 10060] Se produjo un error durante el intento
de conexión ya que la parte conectada no respondió adecuadamente tras un periodo
de tiempo, o bien se produjo un error en la conexión establecida ya que el host c
onectado no ha podido responder>
```

In [2]:

```
#Carga de datos y generación de objeto problem
##############################################################################
problem = tsplib95.load(file)

#Nodos
Nodos = list(problem.get_nodes())

#Aristas
Aristas = list(problem.get_edges())
```

```
---------------------------------------------------------------------------
FileNotFoundError                         Traceback (most recent call last)
Cell In[2], line 3
      1 #Carga de datos y generación de objeto problem
      2 ##############################################################################
######
----> 3 problem = tsplib95.load(file)
      5 #Nodos
      6 Nodos = list(problem.get_nodes())

File ~\anaconda3\envs\PYTHON_IA\Lib\site-packages\tsplib95\loaders.py:16, in load
(filepath, problem_class, special)
      7 def load(filepath, problem_class=None, special=None):
      8     """Load a problem at the given filepath.
      9
     10     :param str filepath: path to a TSPLIB problem file
   (...)     14     :rtype: :class:`~Problem`
     15     """
---> 16     with open(filepath) as f:
     17         return read(f, special=special, problem_class=problem_class)

FileNotFoundError: [Errno 2] No such file or directory: 'swiss42.tsp'
```

In [ ]:
```
Aristas
```

```
NOMBRE: swiss42
TIPO: TSP
COMENTARIO: 42 Staedte Schweiz (Fricker)
DIMENSION: 42
EDGE_WEIGHT_TYPE: EXPLICIT
EDGE_WEIGHT_FORMAT: FULL_MATRIX
EDGE_WEIGHT_SECTION
0 15 30 23 32 55 33 37 92 114 92 110 96 90 74 76 82 72 78 82 159 122 131 206 112 57 28 43 70
  15 0 34 23 27 40 19 32 93 117 88 100 87 75 63 67 71 69 62 63 96 164 132 131 212 106 44 33 5
  30 34 0 11 18 57 36 65 62 84 64 89 76 93 95 100 104 98 57 88 99 130 100 101 179 86 51 4 18
  23 23 11 0 11 48 26 54 70 94 69 75 75 84 84 89 92 89 54 78 99 141 111 109 89 89 11 11 11 54
  32 27 18 11 0 40 20 58 67 92 61 78 65 76 83 89 91 95 43 72 110 141 116 105 190 81 34 19 35
  55 40 57 48 40 0 23 55 96 123 78 75 36 36 66 66 63 95 34 34 137 174 156 129 224 90 15 59 75
  33 19 36 26 20 23 0 45 85 111 75 82 69 60 63 70 71 85 44 52 115 161 136 122 210 91 25 37 54
  37 32 65 54 58 55 45 0 124 149 118 126 113 80 42 42 40 40 87 87 94 158 158 163 242 135 65 6
  92 93 62 70 67 96 85 124 0 28 29 68 63 122 148 155 156 159 67 129 148 78 80 39 129 46 82 65
 114 117 84 94 92 123 111 149 28 0 54 91 88 150 174 181 182 181 95 157 159 50 65 27 102 65 11
  92 88 64 69 61 78 75 118 29 54 0 39 34 99 134 142 141 157 44 110 161 103 109 52 154 22 63 6
 110 100 89 89 78 75 82 126 68 91 39 0 14 80 129 139 135 167 39 98 187 136 148 81 186 28 61 9
  96 87 76 75 65 62 69 113 63 88 34 14 0 72 117 128 124 153 26 88 174 136 142 82 187 32 48 79
  90 75 93 84 76 36 60 80 122 150 99 80 72 0 59 71 63 116 56 25 170 201 189 151 252 104 44 95
  74 63 95 84 83 56 63 42 148 174 134 129 117 59 0 11 8 63 93 35 135 223 195 184 273 146 71 9
```

In [ ]:
```
#Probamos algunas funciones del objeto problem

#Distancia entre nodos
problem.get_weight(0, 1)

#Todas las funciones
#Documentación: https://tsplib95.readthedocs.io/en/v0.6.1/modules.html

#dir(problem)
```

Out[ ]: 15

Funcionas basicas

In [ ]:
```
#Funcionas basicas
##############################################################################

#Se genera una solucion aleatoria con comienzo en en el nodo 0
def crear_solucion(Nodos):
  solucion = [Nodos[0]]
  for n in Nodos[1:]:
    solucion = solucion + [random.choice(list(set(Nodos) - set({Nodos[0]}) - set
  return solucion

#Devuelve la distancia entre dos nodos
def distancia(a,b, problem):
  return problem.get_weight(a,b)

#Devuelve la distancia total de una trayectoria/solucion
def distancia_total(solucion, problem):
  distancia_total = 0
  for i in range(len(solucion)-1):
    distancia_total += distancia(solucion[i] ,solucion[i+1] ,  problem)
  return distancia_total + distancia(solucion[len(solucion)-1] ,solucion[0], prc

sol_temporal = crear_solucion(Nodos)
```

```
distancia_total(sol_temporal, problem), sol_temporal
```

BUSQUEDA ALEATORIA

In [ ]:
```
##############################################################################
# BUSQUEDA ALEATORIA
##############################################################################

def busqueda_aleatoria(problem, N):
  #N es el numero de iteraciones
  Nodos = list(problem.get_nodes())

  mejor_solucion = []
  #mejor_distancia = 10e100                          #Inicializamos con un valor
  mejor_distancia = float('inf')                     #Inicializamos con un valor

  for i in range(N):                                 #Criterio de parada: repetir
    solucion = crear_solucion(Nodos)                 #Genera una solucion aleator
    distancia = distancia_total(solucion, problem)   #Calcula el valor objetivo(a

    if distancia < mejor_distancia:                  #Compara con la mejor obteni
      mejor_solucion = solucion
      mejor_distancia = distancia


  print("Mejor solución:" , mejor_solucion)
  print("Distancia     :" , mejor_distancia)
  return mejor_solucion


#Busqueda aleatoria con 5000 iteraciones
solucion = busqueda_aleatoria(problem, 10000)
```

```
Mejor solución: [0, 11, 36, 16, 31, 17, 37, 27, 30, 1, 6, 33, 22, 35, 34, 2, 8, 2
1, 41, 29, 19, 4, 7, 5, 18, 14, 12, 10, 26, 25, 24, 40, 9, 23, 3, 32, 15, 13, 39,
38, 28, 20]
Distancia     : 3692
```

BUSQUEDA LOCAL

In [ ]:
```
##############################################################################
# BUSQUEDA LOCAL
##############################################################################
def genera_vecina(solucion):
  #Generador de soluciones vecinas: 2-opt (intercambiar 2 nodos) Si hay N nodos
  #Se puede modificar para aplicar otros generadores distintos que 2-opt
  #print(solucion)
  mejor_solucion = []
  mejor_distancia = 10e100
  for i in range(1,len(solucion)-1):          #Recorremos todos los nodos en buc
    for j in range(i+1, len(solucion)):

      #Se genera una nueva solución intercambiando los dos nodos i,j:
      #  (usamos el operador + que para listas en python las concatena) : ej.: [
      vecina = solucion[:i] + [solucion[j]] + solucion[i+1:j] + [solucion[i]] +
```

```
      #Se evalua la nueva solución ...
      distancia_vecina = distancia_total(vecina, problem)

      #... para guardarla si mejora las anteriores
      if distancia_vecina <= mejor_distancia:
        mejor_distancia = distancia_vecina
        mejor_solucion = vecina
  return mejor_solucion


#solucion = [1, 47, 13, 41, 40, 19, 42, 44, 37, 5, 22, 28, 3, 2, 29, 21, 50, 34,
print("Distancia Solucion Incial:" , distancia_total(solucion, problem))


nueva_solucion = genera_vecina(solucion)
print("Distancia Mejor Solucion Local:", distancia_total(nueva_solucion, problem
```

```
Distancia Solucion Incial: 3692
Distancia Mejor Solucion Local: 3383
```

In [ ]:

```
#Busqueda Local:
#  - Sobre el operador de vecindad 2-opt(funcion genera_vecina)
#  - Sin criterio de parada, se para cuando no es posible mejorar.
def busqueda_local(problem):
  mejor_solucion = []

  #Generar una solucion inicial de referencia(aleatoria)
  solucion_referencia = crear_solucion(Nodos)
  mejor_distancia = distancia_total(solucion_referencia, problem)

  iteracion=0             #Un contador para saber las iteraciones que hacemos
  while(1):
    iteracion +=1         #Incrementamos el contador
    #print('#',iteracion)

    #Obtenemos la mejor vecina ...
    vecina = genera_vecina(solucion_referencia)

    #... y la evaluamos para ver si mejoramos respecto a lo encontrado hasta el
    distancia_vecina = distancia_total(vecina, problem)

    #Si no mejoramos hay que terminar. Hemos llegado a un minimo local(según nue
    if distancia_vecina < mejor_distancia:
      #mejor_solucion = copy.deepcopy(vecina)   #Con copia profunda. Las copias
      mejor_solucion = vecina                   #Guarda la mejor solución encont
      mejor_distancia = distancia_vecina

    else:
      print("En la iteracion ", iteracion, ", la mejor solución encontrada es:"
      print("Distancia     :" , mejor_distancia)
      return mejor_solucion

    solucion_referencia = vecina


sol = busqueda_local(problem )
```

```
En la iteracion  35 , la mejor solución encontrada es: [0, 39, 24, 40, 21, 9, 14,
16, 19, 10, 8, 4, 6, 5, 13, 26, 28, 29, 30, 32, 31, 35, 36, 1, 3, 27, 2, 18, 12,
11, 25, 41, 23, 22, 38, 34, 33, 20, 17, 37, 15, 7]
Distancia     : 2040
```

SIMULATED ANNEALING

In [ ]:

```
##############################################################################
# SIMULATED ANNEALING
##############################################################################

#Generador de 1 solucion vecina 2-opt 100% aleatoria (intercambiar 2 nodos)
#Mejorable eligiendo otra forma de elegir una vecina.
def genera_vecina_aleatorio(solucion):

  #Se eligen dos nodos aleatoriamente
  i,j = sorted(random.sample( range(1,len(solucion)) , 2))

  #Devuelve una nueva solución pero intercambiando los dos nodos elegidos al aza
  return solucion[:i] + [solucion[j]] + solucion[i+1:j] + [solucion[i]] + soluci


#Funcion de probabilidad para aceptar peores soluciones
def probabilidad(T,d):
  if random.random() <  math.exp( -1*d / T)  :
    return True
  else:
    return False

#Funcion de descenso de temperatura
def bajar_temperatura(T):
  return T*0.99
```

In [ ]:

```
def recocido_simulado(problem, TEMPERATURA ):
  #problem = datos del problema
  #T = Temperatura

  solucion_referencia = crear_solucion(Nodos)
  distancia_referencia = distancia_total(solucion_referencia, problem)

  mejor_solucion = []             #x* del seudocodigo
  mejor_distancia = 10e100        #F* del seudocodigo


  N=0
  while TEMPERATURA > .0001:
    N+=1
    #Genera una solución vecina
    vecina =genera_vecina_aleatorio(solucion_referencia)

    #Calcula su valor(distancia)
    distancia_vecina = distancia_total(vecina, problem)

    #Si es la mejor solución de todas se guarda(siempre!!!)
    if distancia_vecina < mejor_distancia:
        mejor_solucion = vecina
        mejor_distancia = distancia_vecina
```

```
    #Si la nueva vecina es mejor se cambia
    #Si es peor se cambia según una probabilidad que depende de T y delta(distan
    if distancia_vecina < distancia_referencia or probabilidad(TEMPERATURA, abs(
      #solucion_referencia = copy.deepcopy(vecina)
      solucion_referencia = vecina
      distancia_referencia = distancia_vecina

    #Bajamos la temperatura
    TEMPERATURA = bajar_temperatura(TEMPERATURA)

  print("La mejor solución encontrada es " , end="")
  print(mejor_solucion)
  print("con una distancia total de " , end="")
  print(mejor_distancia)
  return mejor_solucion

sol  = recocido_simulado(problem, 10000000)
```

```
La mejor solución encontrada es [0, 32, 34, 20, 33, 38, 22, 29, 8, 23, 41, 12, 1
0, 9, 39, 30, 31, 35, 36, 18, 11, 25, 21, 24, 40, 26, 13, 16, 15, 19, 5, 3, 2, 4,
6, 14, 37, 17, 7, 1, 27, 28]
con una distancia total de 2151
```